GEMS Action Reference - Table of Contents

- GEMS API Reference
  - Restriction Scopes
  - Triggers
    - DroppedOn
    - KeyPress
    - MouseClick
    - NavBottom
    - NavLeft
    - NavRight
    - NavTop
    - TotalTimePassed
    - ViewTimePassed
  - Conditions
    - KeyBufferContains
    - ObjectInPocketByID
    - ObjectInPocketByName
    - ObjectIsHiddenByID
    - ObjectIsHiddenByName
    - TotalTimePassed
    - VarCountEq
    - VarCountGtEq
    - VarCountLtEq
    - VarExists
    - VarHasString
    - VarLacksString
    - VarValueIs
    - VarValueIsNot
    - ViewTimePassed
    - IsShaded
    - IsNotShaded
  - Actions
    - AllowTake

- ChangeCursor
- ChangeViewImages
- ClearKeyBuffer
- DelVariable
- DisallowTake
- HideMouse
- HideObject
- HidePockets
- HighlightObject
- InputDialog
- PlayBackgroundMusic
- PlaySound
- PlayVideo
- PlayVideoWithin
- PortalTo
- Quit
- RunProgram
- SayText
- SetVariable
- ShowImage
- ShowImageWithin
- ShowMouse
- ShowObject
- ShowPockets
- ShowURL
- StopAllSounds
- StopAllVideos
- StopBackgroundMusic
- StopSound
- StopVideo
- TextBox
- TextDialog
- UnHighlightObject
- ToggleObjectShade
- ShadeObject
- UnshadeObject
- VarAppend
- VarDecrease
- VarIncrease

◦ Parameter Types

GEMS API Reference

Travis L. Seymour, PhD 2026

This document describes all conditions, triggers, and actions available in GEMS (Graphical Environment Management System).

Restriction Scopes

Actions, conditions, and triggers can be restricted to specific contexts:

Scope	Description
`view`	Can be used at the view level
`object`	Can be used on objects within a view
`global`	Can be used in global scripts
`pocket`	Can be used on pocket objects

Triggers

Triggers define events that initiate script execution.

DroppedOn

```
DroppedOn(Object: objnum)
```

Description: Fires when a specific object is dropped onto this object or pocket.

Parameters:

- `Object` (objnum): The ID of the object that must be dropped

Restrictions: object, pocket

KeyPress

```
KeyPress(Key: key)
```

Description: Fires when the specified key is pressed.

Parameters:

- `Key` (key): The key to listen for

Restrictions: view, global

MouseClick

```
MouseClick()
```

Description: Fires when the user clicks on an object or pocket.

Parameters: None

Restrictions: object, pocket

NavBottom

```
NavBottom()
```

Description: Fires when the user clicks in the bottom navigation area of the view.

Parameters: None

Restrictions: view

NavLeft

```
NavLeft()
```

Description: Fires when the user clicks in the left navigation area of the view.

Parameters: None

Restrictions: view

NavRight

```
NavRight()
```

Description: Fires when the user clicks in the right navigation area of the view.

Parameters: None

Restrictions: view

NavTop

```
NavTop()
```

Description: Fires when the user clicks in the top navigation area of the view.

Parameters: None

Restrictions: view

TotalTimePassed

```
TotalTimePassed(Seconds: float)
```

Description: Fires when the specified number of seconds has elapsed since the environment started running.

Parameters:

- `Seconds` (float): Total elapsed time in seconds

Restrictions: global

ViewTimePassed

```
ViewTimePassed(Seconds: float)
```

Description: Fires when the specified number of seconds has elapsed since entering the current view.

Parameters:

- `Seconds` (float): Time in seconds to wait before triggering

Restrictions: view

Conditions

Conditions are evaluated to determine whether associated actions should execute.

KeyBufferContains

```
KeyBufferContains(Keys: value, IgnoreCase: bool)
```

Description: Returns true if the key buffer contains the specified key sequence.

Parameters:

- `Keys` (value): Key sequence to search for
- `IgnoreCase` (bool): If true, comparison is case-insensitive

Restrictions: view, global

ObjectInPocketByID

```
ObjectInPocketByID(Object: objnum)
```

Description: Returns true if the object with the specified ID is currently in one of the pockets.

Parameters:

- `Object` (objnum): ID of the object to check for

Restrictions: view, object, global, pocket

ObjectInPocketByName

```
ObjectInPocketByName(Name: value)
```

Description: Returns true if any object with the specified name is currently in one of the pockets, regardless of its ID.

Parameters:

- `Name` (value): Name of the object to check for

Restrictions: view, object, global, pocket

ObjectIsHiddenByID

```
ObjectIsHiddenByID(Object: objnum)
```

Description: Returns true if the object with the specified ID is currently hidden (not visible) in the current view.

Parameters:

- `Object` (objnum): ID of the object to check

Restrictions: view, object, global, pocket

ObjectIsHiddenByName

```
ObjectIsHiddenByName(Name: value)
```

Description: Returns true if any object with the specified name is currently hidden (not visible) in the current view.

Parameters:

- `Name` (value): Name of the object to check

Restrictions: view, object, global, pocket

TotalTimePassed

```
TotalTimePassed(Seconds: number)
```

Description: Returns true if the specified time has passed since the environment started.

Parameters:

- `Seconds` (number): Time in seconds

Restrictions: view, object, global, pocket

VarCountEq

```
VarCountEq(Count: number)
```

Description: Returns true if the total number of variables equals the specified count.

Parameters:

- `Count` (number): Expected variable count

Restrictions: view, object, global, pocket

VarCountGtEq

```
VarCountGtEq(Count: number)
```

Description: Returns true if the total number of variables is greater than or equal to the specified count.

Parameters:

- `Count` (number): Minimum variable count

Restrictions: view, object, global, pocket

VarCountLtEq

```
VarCountLtEq(Count: number)
```

Description: Returns true if the total number of variables is less than or equal to the specified count.

Parameters:

- `Count` (number): Maximum variable count

Restrictions: view, object, global, pocket

VarExists

```
VarExists(Variable: varname)
```

Description: Returns true if the specified variable exists.

Parameters:

- `Variable` (varname): Name of the variable to check

Restrictions: view, object, global, pocket

VarHasString

```
VarHasString(Variable: varname, SubString: value, Logic: orand,
CaseSensitive: bool)
```

Description: Returns true if the specified variable exists and its string value contains the given substring. SubString may be a comma-separated list of substrings to check for multiple values at once.

Parameters:

- `Variable` (varname): Name of the variable to check
- `SubString` (value): Substring to search for within the variable's value. Use commas to specify multiple substrings (e.g., `"apple, banana, cherry"`)
- `Logic` (orand): Determines how multiple substrings are evaluated. `"or"` (default) returns true if the variable contains **any** of the substrings. `"and"` returns true only if the variable contains **all** of the substrings
- `CaseSensitive` (bool): If `True` (default), the comparison is case-sensitive. If `False`, the comparison is case-insensitive

Restrictions: view, object, global, pocket

Examples:

- `VarHasString("Inventory", "sword")` — true if Inventory contains "sword" (case-sensitive)

- `VarHasString("Inventory", "sword, shield", "or")` — true if Inventory contains "sword" or "shield"
- `VarHasString("Inventory", "sword, shield", "and")` — true only if Inventory contains both "sword" and "shield"
- `VarHasString("Inventory", "Sword", "or", False)` — true if Inventory contains "sword", "Sword", "SWORD", etc.

VarLacksString

```
VarLacksString(Variable: varname, SubString: value, Logic: orand,
CaseSensitive: bool)
```

Description: Returns true if the specified variable does not exist or its string value does not contain the given substring. This is the inverse of VarHasString. SubString may be a comma-separated list of substrings to check for multiple values at once.

Parameters:

- `Variable` (varname): Name of the variable to check
- `SubString` (value): Substring to search for within the variable's value. Use commas to specify multiple substrings (e.g., `"apple, banana, cherry"`)
- `Logic` (orand): Determines how multiple substrings are evaluated. `"or"` (default) returns true if the variable is missing **any** of the substrings. `"and"` returns true only if the variable is missing **all** of the substrings
- `CaseSensitive` (bool): If `True` (default), the comparison is case-sensitive. If `False`, the comparison is case-insensitive

Restrictions: view, object, global, pocket

Examples:

- `VarLacksString("Inventory", "sword")` — true if Inventory does not contain "sword" (case-sensitive)
- `VarLacksString("Inventory", "sword, shield", "or")` — true if Inventory is missing "sword" or "shield" (or both)
- `VarLacksString("Inventory", "sword, shield", "and")` — true only if Inventory is missing both "sword" and "shield"
- `VarLacksString("Inventory", "Sword", "or", False)` — true if Inventory does not contain "sword" in any case

VarValueIs

```
VarValueIs(Variable: varname, Value: value)
```

Description: Returns true if the specified variable exists and equals the given value.

Parameters:

- `Variable` (varname): Name of the variable to check
- `Value` (value): Value to compare against

Restrictions: view, object, global, pocket

VarValueIsNot

```
VarValueIsNot(Variable: varname, Value: value)
```

Description: Returns true if the specified variable does not equal the given value (or doesn't exist).

Parameters:

- `Variable` (varname): Name of the variable to check
- `Value` (value): Value to compare against

Restrictions: view, object, global, pocket

ViewTimePassed

```
ViewTimePassed(Seconds: number)
```

Description: Returns true if the specified time has passed since entering the current view.

Parameters:

- `Seconds` (number): Time in seconds

Restrictions: view, object

IsShaded

```
IsShaded()
```

Description: Returns true if the current object (the one whose actions are being processed) is currently shaded via the ShadeObject or ToggleObjectShade action. This condition takes no parameters and automatically refers to the object in context.

Parameters: None

Restrictions: object, pocket

IsNotShaded

```
IsNotShaded()
```

Description: Returns true if the current object (the one whose actions are being processed) is not currently shaded. This condition takes no parameters and automatically refers to the object in context.

Parameters: None

Restrictions: object, pocket

Actions

Actions are commands that modify the environment state or trigger effects.

AllowTake

```
AllowTake(Object: objnum)
```

Description: Allows an object to be picked up and placed in a pocket.

Parameters:

- `Object` (objnum): ID of the object

Restrictions: view, object, global, pocket

ChangeCursor

```
ChangeCursor(Cursor: cursor)
```

Description: Changes the mouse cursor to the specified style.

Parameters:

- `Cursor` (cursor): Cursor style identifier

Restrictions: (no restrictions)

ChangeViewImages

```
ChangeViewImages(View: viewnum, Foreground: picfile = "", Background:
picfile = "")
```

Description: Changes the Foreground and/or Background images for the specified view. Only valid image file paths will be applied. This is intended to alter images for a view the user may travel to subsequently - if the specified view is the current view, no refresh occurs.

Parameters:

- `View` (viewnum): ID of the view to modify
- `Foreground` (picfile): Optional new foreground image file

- `Background` (picfile): Optional new background image file

Restrictions: view, object, global, pocket

ClearKeyBuffer

```
ClearKeyBuffer()
```

Description: Clears all accumulated keystrokes from the key buffer.

Parameters: None

Restrictions: view, object, global, pocket

DelVariable

```
DelVariable(Variable: varname)
```

Description: Deletes the specified variable.

Parameters:

- `Variable` (varname): Name of the variable to delete

Restrictions: view, object, global, pocket

DisallowTake

```
DisallowTake(Object: objnum)
```

Description: Prevents an object from being picked up.

Parameters:

- `Object` (objnum): ID of the object

Restrictions: view, object, global, pocket

HideMouse

```
HideMouse()
```

Description: Hides the mouse cursor.

Parameters: None

Restrictions: view, object, global, pocket

HideObject

```
HideObject(Object: objnum)
```

Description: Hides a visible object.

Parameters:

- `Object` (objnum): ID of the object to hide

Restrictions: view, object, global, pocket

HidePockets

```
HidePockets()
```

Description: Hides all pocket UI elements.

Parameters: None

Restrictions: view, object, global, pocket

HighlightObject

```
HighlightObject(Object: objnum, Color: fgcolor = "yellow",
LineThickness: number = 4)
```

Description: Draws a visual highlight around the specified object's bounding shape in the current view. The highlight is drawn with solid lines using the specified color and thickness. This highlight does not persist across view changes - if the user leaves and returns to the view, the highlight will be gone. Only objects in the current view can be highlighted; specifying an object from another view will fail gracefully with a log message.

Parameters:

- `Object` (objnum): ID of the object to highlight (must be in current view)
- `Color` (fgcolor): Color for the highlight border (default: yellow)
- `LineThickness` (number): Pen thickness in pixels (default: 4)

Restrictions: view, object, global, pocket

InputDialog

```
InputDialog(Prompt: value, Variable: varname)
```

Description: Displays a dialog prompting the user for input. The entered value is stored in the specified variable.

Parameters:

- `Prompt` (value): Text prompt to display
- `Variable` (varname): Variable to store the input

Restrictions: view, object, global, pocket

PlayBackgroundMusic

```
PlayBackgroundMusic(SoundFile: sndfile, Volume: 01float, Loop: bool)
```

Description: Plays the specified audio file as background music. Only one background music stream can play at a time - calling this while music is playing stops the current music first. Background music persists across view changes and is not affected by StopAllSounds.

Parameters:

- `SoundFile` (sndfile): Path to the audio file
- `Volume` (01float): Volume level (0.0 to 1.0)
- `Loop` (bool): If true, loops the music indefinitely

Restrictions: view, object, global, pocket

PlaySound

```
PlaySound(SoundFile: sndfile, Asynchronous: bool = True, Volume: 01float = 1.0, Loop: bool = False)
```

Description: Plays the specified audio file. Asynchronous playback (the default) returns control immediately; synchronous playback blocks until complete.

Parameters:

- `SoundFile` (sndfile): Path to the audio file
- `Asynchronous` (bool): If true (default), plays without blocking
- `Volume` (01float): Volume level (0.0 to 1.0), default 1.0
- `Loop` (bool): If true, loops the audio continuously (default false)

Restrictions: view, object, global, pocket

PlayVideo

```
PlayVideo(VidFile: vidfile, Start: number, Left: number, Top: number, Volume: 01float, Loop: bool)
```

Description: Plays a video file at the specified position. Right-click to close the video.

Parameters:

- `VidFile` (vidfile): Path to the video file
- `Start` (number): Start time in seconds
- `Left` (number): X position
- `Top` (number): Y position
- `Volume` (01float): Volume level (0.0 to 1.0)
- `Loop` (bool): If true, loops the video

Restrictions: view, object, global, pocket

PlayVideoWithin

```
PlayVideoWithin(VidFile: vidfile, Start: number, WithinObject: objnum,
Volume: 01float, Loop: bool)
```

Description: Plays a video within the bounds of a specified object. The video is scaled to fit the object's dimensions.

Parameters:

- `VidFile` (vidfile): Path to the video file
- `Start` (number): Start time in seconds
- `WithinObject` (objnum): ID of the object to play within
- `Volume` (01float): Volume level (0.0 to 1.0)
- `Loop` (bool): If true, loops the video

Restrictions: view, object, global, pocket

PortalTo

```
PortalTo(View: viewnum, VidFile: vidfile = "", Delay: float = 0.0)
```

Description: Navigates to the specified view. If a video file is provided, plays it as a fullscreen transition before changing views. Right-clicking the video skips to the destination view immediately. If Delay is greater than zero, the portal waits the specified number of

seconds before executing; during this delay, the mouse is hidden and no actions are triggered.

Parameters:

- `View` (viewnum): ID of the destination view
- `VidFile` (vidfile): Optional video file for transition effect
- `Delay` (float): Optional delay in seconds before portal executes (default: 0.0). During the delay, the mouse is hidden and actions are disabled.

Restrictions: view, object, global, pocket

Quit

```
Quit()
```

Description: Terminates the GEMS environment.

Parameters: None

Restrictions: view, object, global, pocket

RunProgram

```
RunProgram(Application: exefile, Parameters: value)
```

Description: Launches an external application with the specified parameters.

Parameters:

- `Application` (exefile): Path to the executable
- `Parameters` (value): Command-line parameters

Restrictions: view, object, global, pocket

SayText

```
SayText(Message: value)
```

Description: Uses text-to-speech to speak the specified message.

Parameters:

- `Message` (value): Text to speak

Restrictions: view, object, global, pocket

SetVariable

```
SetVariable(Variable: varname, Value: value)
```

Description: Creates or updates a variable with the specified value.

Parameters:

- `Variable` (varname): Name of the variable
- `Value` (value): Value to assign

Restrictions: view, object, global, pocket

ShowImage

```
ShowImage(ImageFile: picfile, Left: number, Top: number, Duration:
float, Clickthrough: bool)
```

Description: Displays an image at the specified position for a given duration.

Parameters:

- `ImageFile` (picfile): Path to the image file
- `Left` (number): X position
- `Top` (number): Y position

- `Duration` (float): How long to display (seconds), 0 for permanent
- `Clickthrough` (bool): If true, clicks pass through the image

Restrictions: view, object

ShowImageWithin

```
ShowImageWithin(ImageFile: picfile, Left: number, Top: number, Duration: float, Clickthrough: bool, WithinObject: number, HideTarget: bool, Stretch: bool)
```

Description: Displays an image within the bounds of a specified object.

Parameters:

- `ImageFile` (picfile): Path to the image file
- `Left` (number): X offset within the object
- `Top` (number): Y offset within the object
- `Duration` (float): How long to display (seconds)
- `Clickthrough` (bool): If true, clicks pass through
- `WithinObject` (number): ID of the object to display within
- `HideTarget` (bool): If true, hides the target object
- `Stretch` (bool): If true, stretches image to fit object bounds

Restrictions: view, object

ShowMouse

```
ShowMouse()
```

Description: Shows the mouse cursor if hidden.

Parameters: None

Restrictions: view, object, global, pocket

ShowObject

```
ShowObject(Object: objnum)
```

Description: Makes a hidden object visible.

Parameters:

- `Object` (objnum): ID of the object to show

Restrictions: view, object, global, pocket

ShowPockets

```
ShowPockets()
```

Description: Shows all pocket UI elements.

Parameters: None

Restrictions: view, object, global, pocket

ShowURL

```
ShowURL(URL: value)
```

Description: Opens the specified URL in the default web browser.

Parameters:

- `URL` (value): URL to open

Restrictions: view, object, global, pocket

StopAllSounds

```
StopAllSounds()
```

Description: Stops all currently playing sound effects. Does not affect background music.

Parameters: None

Restrictions: view, object, global, pocket

StopAllVideos

```
StopAllVideos()
```

Description: Stops all currently playing videos.

Parameters: None

Restrictions: view, object, global, pocket

StopBackgroundMusic

```
StopBackgroundMusic()
```

Description: Stops the currently playing background music.

Parameters: None

Restrictions: view, object, global, pocket

StopSound

```
StopSound(SoundFile: sndfile)
```

Description: Stops playback of the specified sound file if currently playing.

Parameters:

- `SoundFile` (sndfile): Path to the audio file to stop

Restrictions: view, object, global, pocket

StopVideo

```
StopVideo(VideoFile: vidfile)
```

Description: Stops playback of the specified video file.

Parameters:

- `VideoFile` (vidfile): Path to the video file to stop

Restrictions: view, object, global, pocket

TextBox

```
TextBox(Message: value, Left: number, Top: number, Duration: float,
FontColor: fgcolor, BackColor: bgcolor, FontSize: fontsize, Bold: bool)
```

Description: Displays a text box with the specified message and styling.

Parameters:

- `Message` (value): Text to display
- `Left` (number): X position
- `Top` (number): Y position
- `Duration` (float): How long to display (seconds)
- `FontColor` (fgcolor): Text color
- `BackColor` (bgcolor): Background color
- `FontSize` (fontsize): Font size
- `Bold` (bool): If true, uses bold text

Restrictions: view, object, global, pocket

TextDialog

```
TextDialog(Message: value)
```

Description: Displays a modal dialog box with the specified message.

Parameters:

- `Message` (value): Text to display

Restrictions: view, object, global, pocket

UnHighlightObject

```
UnHighlightObject(Object: objnum)
```

Description: Removes any highlight previously applied to the specified object via HighlightObject. If no highlight exists on the object, this action has no effect. Only objects in the current view can be referenced; specifying an object from another view will fail gracefully with a log message.

Parameters:

- `Object` (objnum): ID of the object to remove highlight from (must be in current view)

Restrictions: view, object, global, pocket

ToggleObjectShade

```
ToggleObjectShade(NameSubstring: value, Color: color)
```

Description: Toggles shading on all objects across all views whose name contains the specified substring. If a matching object is already shaded, the shade is removed. If not shaded, a filled shade is drawn over the object's bounding shape using the specified color

with alpha for transparency. Shading persists across view changes. Use the IsShaded/IsNotShaded conditions to check whether the current object is shaded.

Parameters:

- `NameSubstring` (value): Substring to match against object names. All objects whose name contains this substring will be affected.
- `Color` (color): Color for the shade fill, including alpha for transparency (e.g., `['Red',255,0,0,128]`)

Restrictions: view, object, global, pocket

Example:

- `ToggleObjectShade("Lamp", "['Yellow',255,255,0,100]")` — Toggles shading on all objects with "Lamp" in their name

---

ShadeObject

```
ShadeObject(NameSubstring: value, Color: color)
```

Description: Draws a filled shade over all objects across all views whose name contains the specified substring. The shade uses the specified color with alpha for transparency and persists across view changes until removed with UnshadeObject. If an object is already shaded, this action updates the shade color.

Parameters:

- `NameSubstring` (value): Substring to match against object names. All objects whose name contains this substring will be shaded.
- `Color` (color): Color for the shade fill, including alpha for transparency (e.g., `['Red',255,0,0,128]`)

Restrictions: view, object, global, pocket

UnshadeObject

```
UnshadeObject(NameSubstring: value)
```

Description: Removes any shade from all objects across all views whose name contains the specified substring. If no matching objects are shaded, this action has no effect.

Parameters:

- `NameSubstring` (value): Substring to match against object names. All matching objects will have their shading removed.

Restrictions: view, object, global, pocket

VarAppend

```
VarAppend(Variable: varname, Text: value)
```

Description: Appends the specified text to the string value of the variable. If the variable does not exist, it will be created with the text as its value.

Parameters:

- `Variable` (varname): Name of the variable to append to
- `Text` (value): Text to append to the variable's value

Restrictions: view, object, global, pocket

VarDecrease

```
VarDecrease(Variable: varname)
```

Description: Decreases the value of the specified variable by 1. If the variable does not exist or has a non-numeric value, it will be created and set to 0.

Parameters:

- `Variable` (varname): Name of the variable to decrement

Restrictions: view, object, global, pocket

VarIncrease

```
VarIncrease(Variable: varname)
```

Description: Increases the value of the specified variable by 1. If the variable does not exist or has a non-numeric value, it will be created and set to 1.

Parameters:

- `Variable` (varname): Name of the variable to increment

Restrictions: view, object, global, pocket

Parameter Types

Type	Description
`bool`	Boolean (True/False)
`bgcolor`	Background color
`color`	Color with alpha (e.g., `['Red',255,0,0,128]`)
`cursor`	Cursor style
`exefile`	Executable file path
`fgcolor`	Foreground color
`float`	Decimal number
`fontsize`	Font size
`key`	Keyboard key

Type	Description
`number`	Integer value
`objnum`	Object ID selector
`01float`	Float between 0.0 and 1.0
`orand`	Logic operator ("or" or "and")
`picfile`	Image file path
`sndfile`	Audio file path
`value`	Text string
`varname`	Variable name
`vidfile`	Video file path
`viewnum`	View ID selector